

GIÁO TRÌNH THỰC CHIẾN

MẬT MÃ HỌC CHO REVERSE ENGINEERING

Từ bitwise và magic constants đến trích xuất key
và viết script giải mã

RE • MALWARE • CTF

LỜI NÓI ĐẦU

Mật mã học dưới góc nhìn của Reverse Engineer

Đối với lập trình viên, mật mã học thường xuất hiện dưới dạng một lời gọi thư viện. Đối với Reverse Engineer hoặc Malware Analyst, bài toán hoàn toàn khác: tên hàm có thể đã bị strip, dữ liệu có thể được dựng tại runtime, và thuật toán chuẩn có thể bị sửa hằng số để tránh nhận diện tự động.

Vì vậy, mục tiêu của tài liệu này không phải là chứng minh mọi định lý mật mã. Mục tiêu là giúp người học nhìn vào pseudocode, assembly, vùng nhớ và các hằng số để trả lời bốn câu hỏi thực dụng:

1. Đây là mã hóa, băm, checksum hay chỉ là phép biến đổi dữ liệu?
2. Thuật toán thuộc họ nào và dấu hiệu nào củng cố giả thuyết đó?
3. Key, IV, nonce, bảng thay thế hoặc state nằm ở đâu?
4. Làm thế nào mô phỏng hoặc đảo ngược nó bằng một script ngắn?

Nguyên tắc đọc

Không kết luận chỉ từ một magic constant. Hãy ghép ít nhất ba nhóm bằng chứng: hằng số, hình dạng vòng lặp và cách dữ liệu đi vào/đi ra.

CÁCH DÙNG TÀI LIỆU

- Đọc Phần 1 để chắc các thao tác bit và phép biến đổi đảo được.
- Dùng Phần 2-4 như sổ tay nhận diện thuật toán khi làm việc trong IDA/Ghidra.
- Áp dụng Phần 5 như checklist khi xử lý một binary thật hoặc một challenge CTF.

MỤC LỤC

01 Nền tảng toán học và thao tác bitwise	4
02 Mật mã khóa đối xứng	13
03 Hàm băm, checksum và API hashing	23
04 Mật mã khóa công khai	29
05 Quy trình reverse crypto thực chiến	49
Phụ lục Cheat sheet nhận diện và mẫu script	58

Nhóm	Câu hỏi cần trả lời	Dấu hiệu thường gặp
Encoding	Biểu diễn hay che giấu?	Bảng chữ cái, padding, chiều dài tăng có quy luật
Checksum	Phát hiện lỗi hay xác minh toàn vẹn?	State nhỏ, bảng tra, không dùng key
Hash	Ảnh xạ một chiều?	Padding, nhiều vòng, digest cố định
Encryption	Có key và có thể giải mã?	Key schedule, IV/nonce, encrypt/decrypt

Phạm vi

Tài liệu tập trung vào nhận diện và phân tích hợp pháp trong CTF, phòng thí nghiệm, malware analysis và phần mềm do bạn được phép kiểm tra.

PHẦN 1

NỀN TẢNG TOÁN HỌC & BITWISE

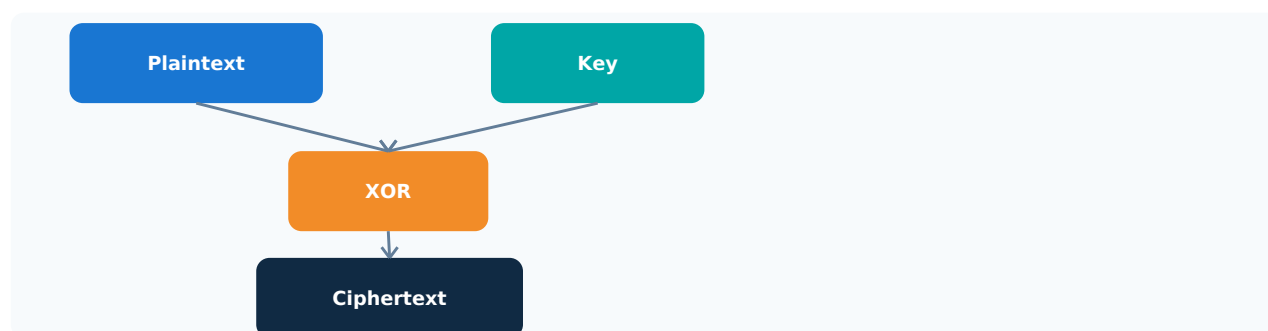
Những phép toán nhỏ xuất hiện lặp đi lặp lại trong crypto, packer và obfuscation.

1.1. XOR và Repeated-key XOR

XOR ($\oplus$) tương đương phép cộng modulo 2 trên từng bit. Nó được dùng rất nhiều vì rẻ, nhanh và có tính tự nghịch đảo.

$$A \oplus B = C \Rightarrow C \oplus B = A$$

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0



Trong repeated-key XOR, byte khóa được chọn theo $\text{key}[i \bmod \text{key_len}]$. Nếu biết một đoạn plaintext, ta có thể XOR nó với ciphertext để lộ đoạn key tương ứng.

```
def xor_repeat(data: bytes, key: bytes) -> bytes:
    return bytes(b ^ key[i % len(key)] for i, b in enumerate(data))

cipher = bytes.fromhex("2b0d151e0a")
print(xor_repeat(cipher, b"KEY"))
```

Phá repeated-key XOR thường gồm hai bước:

1. Ước lượng độ dài khóa bằng Hamming distance chuẩn hóa giữa nhiều cặp block.
2. Chuyển vị ciphertext thành L cột; mỗi cột trở thành một bài toán single-byte XOR và được chấm điểm theo tần suất ngôn ngữ.

Dấu hiệu trong decompiler

Một vòng lặp XOR từng byte, chỉ số khóa quay về 0 hoặc dùng phép chia dư theo độ dài khóa. Nếu key_len là lũy thừa của 2, phép modulo có thể biến thành AND.

1.2. Cộng/trừ modulo 256

Với một byte, phép cộng tự nhiên bị cuộn trong miền 0..255. Trong C, hành vi này thường xuất hiện khi kết quả được ép về `uint8_t`.

$$c = (p + k) \bmod 256 \Leftrightarrow p = (c - k) \bmod 256$$

```
# Python không tự tràn byte, vì vậy phải dùng & 0xff
enc = bytes((b + 17) & 0xff for b in data)
dec = bytes((b - 17) & 0xff for b in enc)
```

Bẫy dấu

Decompiler có thể hiển thị char có dấu. Khi mô phỏng bằng Python, hãy chuẩn hóa sau mỗi phép toán bằng `& 0xFF` hoặc `& 0xFFFFFFFF` đúng theo độ rộng toán hạng.

1.3. Shift và Rotate

Shift làm mất các bit bị đẩy ra ngoài; rotate đưa chúng quay về đầu kia. Vì rotate bảo toàn thông tin nên nó xuất hiện thường xuyên trong hash, ARX cipher và custom crypto.

Phép	Hiệu ứng	Khả năng đảo
$x \ll n$	Dịch trái, thêm 0 ở phải	Không tự đảo được nếu bit cao đã mất
$x \gg n$	Dịch phải; logical hoặc arithmetic	Không tự đảo được
$\text{ROL}(x, n)$	Quay trái trong word cố định	Đảo bằng $\text{ROR}(x, n)$
$\text{ROR}(x, n)$	Quay phải trong word cố định	Đảo bằng $\text{ROL}(x, n)$

```
static inline uint32_t rol32(uint32_t x, unsigned n) {
    n &= 31;
    return (x << n) | (x >> ((32 - n) & 31));
}
```

IDA thường rút gọn mẫu trên thành `__ROL4__(x, n)`. Trong assembly x86/x64, bạn có thể thấy trực tiếp `rol` hoặc `ror`.

1.4. Endianness

Endianness không thay đổi giá trị logic, nhưng thay đổi thứ tự byte trong bộ nhớ. Trên Windows x86/x64, số nhiều byte được lưu theo little-endian.

Giá trị 32-bit	Bytes trong bộ nhớ	Cách đọc
0x12345678	78 56 34 12	Little-endian
0x12345678	12 34 56 78	Big-endian / network order

Đây là lý do một magic constant đôi khi xuất hiện đảo byte trong hex view. Khi trích key hoặc ciphertext từ debugger, phải xác định xem bạn đang sao chép chuỗi byte hay giá trị integer.

1.5. Hoán vị và S-box

Hoán vị chỉ đổi vị trí phần tử; muốn đảo, hãy xây dựng inverse permutation. S-box thay thế giá trị bằng bảng tra và tạo tính phi tuyến. Một S-box có thể đảo được nếu mọi đầu ra là duy nhất.

```
def inverse_permutation(p):
    inv = [0] * len(p)
    for out_pos, in_pos in enumerate(p):
        inv[in_pos] = out_pos
    return inv

def invert_sbox(sbox):
    inv = [0] * 256
    for x, y in enumerate(sbox):
        inv[y] = x
    return inv
```

Dấu hiệu S-box

Một mảng 256 byte, được truy cập bằng chỉ số phụ thuộc dữ liệu: `movzx eax, byte ptr [base + index]`. Tuy nhiên, một bảng 256 phần tử cũng có thể là CRC table; hãy xem dữ liệu sau lookup được XOR, rotate hay dùng làm chỉ số tiếp theo.

1.6. XOR: ví dụ byte-by-byte hoàn chỉnh

Giả sử plaintext là FLAG và key một byte là 0x23. Ta không nên nhìn XOR như một phép toán mơ hồ; hãy viết từng byte ở dạng hex để thấy chính xác dữ liệu biến đổi thế nào.

Vị trí	Plaintext	Mã ASCII	Key	Ciphertext
0	F	0x46	0x23	0x65
1	L	0x4C	0x23	0x6F
2	A	0x41	0x23	0x62
3	G	0x47	0x23	0x64

Ciphertext nhận được là 65 6F 62 64. Để giải mã, XOR lại mỗi byte với 0x23. Không cần một thuật toán decrypt riêng vì XOR tự nghịch đảo.

Known-plaintext attack

Nếu biết hoặc đoán được một đoạn plaintext P nằm tại cùng vị trí với ciphertext C, ta thu được keystream $K = P \oplus C$. Với repeated-key XOR, đoạn keystream này có thể tiết lộ toàn bộ key nếu đoạn đã biết dài ít nhất bằng key.

```
known = b"MZ" # Header PE thường gặp
cipher_prefix = bytes.fromhex("1f3f")
key_prefix = bytes(a ^ b for a, b in zip(known, cipher_prefix))
print(key_prefix.hex())
```

Khi reverse malware

Các plaintext đoán được thường là MZ, PK, {, http, tên field JSON hoặc header cấu hình. Đây là cách kiểm tra nhanh giả thuyết XOR trước khi phân tích cả hàm.

1.7. Hamming distance và tìm độ dài khóa XOR

Hamming distance đếm số bit khác nhau giữa hai chuỗi byte cùng độ dài. Ví dụ XOR hai block rồi đếm số bit 1. Với repeated-key XOR, các block được mã hóa bằng cùng chu kỳ key có xu hướng giữ lại cấu trúc thống kê tương tự; khoảng cách chuẩn hóa có thể giúp xếp hạng key length.

$$D(X,Y) = \text{popcount}(X \oplus Y)$$

```
def hamming(a: bytes, b: bytes) -> int:
    assert len(a) == len(b)
    return sum((x ^ y).bit_count() for x, y in zip(a, b))

def score_key_length(ct: bytes, k: int, blocks=8):
    chunks = [ct[i*k:(i+1)*k] for i in range(blocks)]
    chunks = [x for x in chunks if len(x) == k]
    pairs = list(zip(chunks, chunks[1:]))
    return sum(hamming(a, b) / k for a, b in pairs) / len(pairs)

ranking = sorted((score_key_length(cipher, k), k) for k in range(2, 41))
print(ranking[:5])
```

Không chọn key length chỉ vì nó đứng đầu một lần. Hãy dùng nhiều block, kiểm tra cả bội số của key length và xác minh bằng chất lượng plaintext sau khi giải từng cột single-byte XOR.

Chấm điểm single-byte XOR

Một cách đơn giản là thưởng ký tự in được, dấu cách và chữ cái; phạt byte điều khiển. Với dữ liệu tiếng Anh có thể dùng tần suất chữ. Với config malware, tập ký tự kỳ vọng nên gồm ASCII, dấu phân cách, URL, path và JSON.

1.8. AND, OR, NOT và bit mask

Các phép này thường không phải crypto độc lập, nhưng chúng định hình cách state được cắt, ghép và chuẩn hóa. Reverse Engineer phải đọc được mask ngay khi nhìn thấy.

Mẫu	Ý nghĩa	Ví dụ
$x \& \text{mask}$	Giữ các bit có mask=1	$x \& 0xFF$ lấy byte thấp
$x \text{mask}$	Bật các bit có mask=1	$\text{flags} 0x04$ bật bit 2
$x \wedge \text{mask}$	Đảo các bit có mask=1	$x \wedge 0x20$ đổi case ASCII trong vài trường hợp
$\sim x$	Đảo toàn bộ bit	Cần mask lại theo 8/32/64-bit
$x \& (n-1)$	Modulo n khi n là lũy thừa 2	$i \& 0xFF$ tương đương $i \bmod 256$

Bit field

Một byte có thể chứa nhiều cờ. Ví dụ `flags = 0b10100100`: kiểm tra bit 5 bằng `(flags >> 5) & 1`. Decompiler thường biến thao tác này thành biểu thức khó đọc nếu type bị suy đoán sai.

```
def get_bits(x, start, width):
    return (x >> start) & ((1 << width) - 1)

opcode = get_bits(word, 12, 4) # lấy 4 bit từ vị trí 12
length = get_bits(word, 0, 12) # lấy 12 bit thấp
```

Sai lầm hay gặp

Trong Python, `~0x12` cho ra số âm vì integer không giới hạn độ rộng. Muốn mô phỏng NOT 8-bit phải dùng `(~x) & 0xFF`; NOT 32-bit dùng `& 0xFFFFFFFF`.

1.9. Encoding không phải encryption

Hex, Base64 và Base32 chỉ chuyển byte thành ký tự để lưu truyền. Không có secret key và bất kỳ ai cũng giải mã được. Trong malware, encoding thường bọc bên ngoài ciphertext hoặc payload nén, nên phải bóc từng lớp.

Định dạng	Alphabet / dấu hiệu	Quan hệ chiều dài
Hex	0-9, a-f; thường có 0x	1 byte → 2 ký tự
Base64	A-Z a-z 0-9 + / và = padding	3 byte → 4 ký tự
Base64 URL-safe	- và _ thay + và /	Tương tự Base64
Base32	A-Z 2-7 và = padding	5 byte → 8 ký tự
ROT13	Chỉ đổi chữ cái 13 vị trí	Chiều dài giữ nguyên

```
import base64

raw = bytes.fromhex("4d5a9000")
wrapped = base64.b64encode(raw)
assert base64.b64decode(wrapped) == raw
print(wrapped) # b'TVqQAA=='
```

Pipeline nhiều lớp

Một chuỗi có thể đi qua Base64 decode → XOR → zlib decompress → PE. Sau mỗi lớp, hãy kiểm tra magic bytes, entropy, độ dài và định dạng thay vì đoán lớp tiếp theo.

1.10. RLE, nén và entropy

Run-Length Encoding lưu các cặp (count,value). Ví dụ AAAAABBCC có thể thành (5,A) (2,B) (2,C). RLE đảo được nhưng không phải mã hóa; nó chỉ hiệu quả khi dữ liệu có nhiều chuỗi lặp.

```
def rle_decode(pairs):
    out = bytearray()
    for count, value in pairs:
        out.extend([value] * count)
    return bytes(out)

print(rle_decode([(5, ord('A')), (2, ord('B')), (2, ord('C'))]))
```

Entropy Shannon ước lượng mức phân tán của byte. Dữ liệu nén hoặc mã hóa thường có entropy cao, nhưng entropy cao không chứng minh đó là encryption. Một bảng random, ảnh nén hoặc section chứa resource cũng có thể tương tự.

$$H(X) = - \sum p(x) \log_2 p(x)$$

Quan sát	Giả thuyết hợp lý	Bước xác minh
Entropy thấp + nhiều byte 00	Dữ liệu thô / cấu trúc	Tìm header, record, string
Entropy cao toàn section	Packed/compressed/encrypted	Tìm unpack/decrypt loop hoặc API
Entropy cao chỉ ở một blob	Payload/config/resource	Theo xref tới code đọc blob
Sau transform xuất hiện magic	Đã bóc đúng một lớp	Parse định dạng và tiếp tục pipeline

1.11. PRNG: LCG, rand(), seed và khả năng dự đoán

Linear Congruential Generator tạo state tiếp theo bằng công thức $x_{n+1} = (a \cdot x_n + c) \bmod m$. Nó nhanh nhưng không an toàn cho crypto. Nếu seed lấy từ thời gian và khoảng tìm kiếm nhỏ, có thể thử lại toàn bộ seed.

$$x_{n+1} = (a \cdot x_n + c) \bmod m$$

```
def lcg(seed, a=1103515245, c=12345, m=2**31):
    x = seed
    while True:
        x = (a*x + c) % m
        yield x

g = lcg(1700000000)
print([next(g) for _ in range(4)])
```

Nhận diện trong assembly

- Một biến state được nhân với hằng số lớn, cộng hằng số rồi mask/modulo.
- State toàn cục hoặc nằm trong context struct và được cập nhật sau mỗi lần gọi.
- Seed đến từ time, GetTickCount, PID/TID hoặc QueryPerformanceCounter.

Khi reverse, ghi rõ generator dùng toàn bộ state hay chỉ lấy một số bit cao/thấp. Hàm C rand() khác nhau giữa runtime; không được mặc định mọi binary dùng cùng constants.

Phân biệt nonce và randomness

Nonce chỉ cần không lặp trong nhiều thiết kế; key cần không thể đoán. Dùng timestamp làm nonce có thể chấp nhận trong vài giao thức, nhưng dùng timestamp làm key/seed bí mật là cực yếu.

PHẦN 2

MẬT MÃ KHÓA ĐỐI XỨNG

Nhận diện cấu trúc, key schedule, state và ranh giới giữa thuật toán chuẩn với biến thể tùy chỉnh.

2.1. RC4: KSA và PRGA

RC4 duy trì một hoán vị S gồm 256 byte. Giai đoạn KSA trộn S bằng key; PRGA tiếp tục swap các phần tử để sinh keystream rồi XOR với dữ liệu.

Giai đoạn	Mẫu hành vi	Giá trị cần quan sát
KSA	Khởi tạo S[i]=i rồi swap 256 lần	Key, key_len, S sau KSA
PRGA	Tăng i, cập nhật j, swap, lấy S[S[i]+S[j]]	i, j, keystream byte
XOR	out[n] = in[n] XOR stream	Input/output buffer, length

```
def rc4(data: bytes, key: bytes) -> bytes:
    S = list(range(256))
    j = 0
    for i in range(256):
        j = (j + S[i] + key[i % len(key)]) & 0xff
        S[i], S[j] = S[j], S[i]
    i = j = 0
    out = bytearray()
    for b in data:
        i = (i + 1) & 0xff
        j = (j + S[i]) & 0xff
        S[i], S[j] = S[j], S[i]
        out.append(b ^ S[(S[i] + S[j]) & 0xff])
    return bytes(out)
```

Nhận diện chắc tay

Bộ ba bằng chứng mạnh nhất là: mảng 256 byte được khởi tạo 0..255, swap phụ thuộc key trong đúng 256 vòng, và vòng thứ hai phát sinh byte rồi XOR dữ liệu.

2.2. TEA, XTEA và XXTEA

Họ TEA nổi bật nhờ hằng số delta 0x9E3779B9. Nhưng compiler có thể biểu diễn delta dưới dạng số âm 0x61C88647 khi giải mã bằng phép trừ.

Thuật toán	Cấu trúc	Dấu hiệu phân biệt
TEA	Block 64-bit, key 128-bit, thường 32 vòng	Hai word cập nhật đối xứng; shift 4 và 5
XTEA	Block 64-bit, key 128-bit	Chọn key bằng $\text{sum} \& 3$ và $(\text{sum} \gg 11) \& 3$
XXTEA	Mảng word có độ dài biến đổi	Vòng ngoài $6 + 52/n$; phụ thuộc phần tử trước/sau

```
// Lỗi TEA điển hình
for (int i = 0; i < 32; ++i) {
    sum += 0x9E3779B9;
    v0 += ((v1 << 4) + k0) ^ (v1 + sum) ^ ((v1 >> 5) + k1);
    v1 += ((v0 << 4) + k2) ^ (v0 + sum) ^ ((v0 >> 5) + k3);
}
```

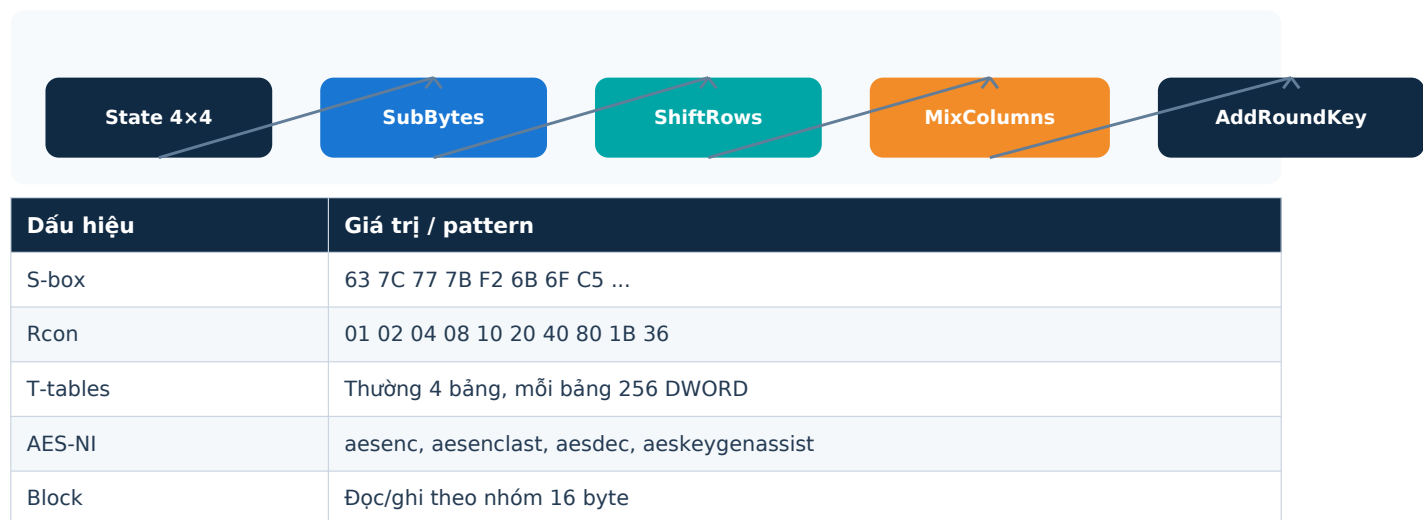
Khi viết script đảo, phải xác định chính xác thứ tự cập nhật. Trong TEA, v1 của mỗi vòng sử dụng v0 đã được cập nhật. Khi decrypt, vì thế phải đảo v1 trước rồi mới đảo v0.

Custom TEA

Malware/CTF có thể đổi delta, số vòng, thứ tự cộng-XOR hoặc endianness. Đừng chỉ tìm hằng số; hãy nhìn cấu trúc hai half-word và chuỗi shift/add/xor lặp lại.

2.3. AES

AES làm việc trên block 16 byte và có 10/12/14 round tương ứng key 128/192/256-bit. Một round chuẩn gồm SubBytes, ShiftRows, MixColumns và AddRoundKey; round cuối bỏ MixColumns.



Trong Windows malware, AES thường xuất hiện qua hai con đường: triển khai nội bộ hoặc API như BCrypt/CNG, CryptoAPI, OpenSSL. Với trường hợp gọi thư viện, reverse toàn bộ AES thường không cần thiết; breakpoint tại API để thu key, IV, mode, input và output sẽ nhanh hơn.

2.4. Mode of operation và IV/nonce

Mode	Nhận diện	Sai lầm triển khai đáng chú ý
ECB	Mỗi block độc lập	Block plaintext giống nhau cho ciphertext giống nhau
CBC	Cipher block trước XOR vào plaintext sau	IV cố định hoặc padding oracle
CTR	Mã hóa counter rồi XOR	Tái sử dụng nonce làm tái sử dụng keystream
GCM	CTR + authentication tag	Nonce reuse có thể phá bảo mật nghiêm trọng

Key không đủ

Để giải mã đúng, thường còn cần mode, IV/nonce, padding, byte order và cách dữ liệu được đóng gói. Khi debug, hãy ghi lại toàn bộ tham số chứ không chỉ key.

2.5. RC4: diễn giải từng vòng KSA

KSA bắt đầu với permutation đồng nhất $S=[0,1,\dots,255]$. Biến j tích lũy giá trị của $S[i]$ và key . Mỗi vòng swap $S[i]$ với $S[j]$, khiến ảnh hưởng của key lan ra toàn bảng.

$$j = (j + S[i] + \text{key}[i \bmod \text{key_len}]) \bmod 256$$

i	j trước	S[i]	key[i mod L]	j sau	Thao tác
0	0	0	K	75	swap S[0],S[75]
1	75	1	e	177	swap S[1],S[177]
2	177	2	y	44	swap S[2],S[44]

Bảng trên chỉ minh họa ba vòng đầu với key ASCII Key. Sau 256 vòng, S là một permutation phụ thuộc key . PRGA không khởi tạo lại S ; nó tiếp tục thay đổi bảng trong quá trình phát keystream.

Điểm đặt breakpoint tốt

- Ngay sau vòng lặp 256 lần: dump 256 byte của S để xác nhận KSA.
- Ngay trước XOR output: log keystream byte, input byte và output byte.
- Tại caller: xác định key pointer, key length, data pointer và data length.

Biến thể

RC4-drop[N] bỏ N byte keystream đầu; malware có thể đảo thứ tự cập nhật hoặc khởi tạo S khác. Script RC4 chuẩn sẽ không khớp nếu bỏ qua biến thể này.

2.6. TEA: từ mã hóa sang giải mã

TEA mã hóa hai word 32-bit v_0, v_1 bằng 32 round và bốn word key. Vì mọi phép cộng/trừ diễn ra modulo 2^{32} , Python phải mask sau từng bước. Giải mã thực hiện phép toán theo thứ tự ngược.

```
def tea_decrypt_block(block8: bytes, key16: bytes) -> bytes:
    import struct
    v0, v1 = struct.unpack('<2I', block8)
    k0, k1, k2, k3 = struct.unpack('<4I', key16)
    delta = 0x9E3779B9
    total = (delta * 32) & 0xffffffff
    for _ in range(32):
        v1 = (v1 - (((v0 << 4) + k2) ^ (v0 + total) ^
                    ((v0 >> 5) + k3))) & 0xffffffff
        v0 = (v0 - (((v1 << 4) + k0) ^ (v1 + total) ^
                    ((v1 >> 5) + k1))) & 0xffffffff
        total = (total - delta) & 0xffffffff
    return struct.pack('<2I', v0, v1)
```

Cách kiểm tra script

1. Lấy một block plaintext tự chọn, chạy qua hàm encrypt trong binary.
2. Ghi lại ciphertext, key và byte order ở runtime.
3. Chạy script decrypt; phải thu đúng plaintext byte-for-byte.
4. Nếu sai, kiểm tra thứ tự key words, endian, delta, số round và biểu thức parentheses.

XTEA khác TEA ở lịch chọn key. XXTEA không xử lý các block 8 byte độc lập mà trộn cả mảng word. Vì vậy không được thấy delta rồi lập tức dùng script TEA chuẩn.

2.7. AES State và bốn phép biến đổi

AES sắp 16 byte input vào ma trận 4×4 theo cột. Đây là nguồn của nhiều nhầm lẫn: thứ tự hiển thị ma trận trong tài liệu không nhất thiết giống thứ tự byte liên tiếp trong debugger.

Bước	Tác dụng	Tính chất RE cần nhớ
SubBytes	Thay mỗi byte qua AES S-box	Lookup phi tuyến; S-box 256 byte
ShiftRows	Quay hàng 0/1/2/3 vị trí	Chỉ hoán vị byte, không đổi giá trị
MixColumns	Nhân mỗi cột trong $GF(2^8)$	Xtime, XOR, constants 02/03
AddRoundKey	XOR state với round key	Bước duy nhất trực tiếp đưa key vào state

$GF(2^8)$ và xtime

Trong MixColumns, nhân một byte với 2 có thể được triển khai bằng shift trái; nếu bit cao ban đầu bằng 1 thì XOR với 0x1B. Mẫu này thường xuất hiện trong AES triển khai không dùng table.

```
def xtime(x):
    return ((x << 1) ^ (0x1B if x & 0x80 else 0)) & 0xff

def mul3(x):
    return xtime(x) ^ x
```

AES tableless

Không thấy S-box rõ ràng không có nghĩa là không phải AES. Compiler có thể vector hóa, dùng AES-NI, bitslicing hoặc gọi thư viện hệ thống.

2.8. AES key schedule

AES không dùng trực tiếp cùng 16 byte key cho mọi round. Key expansion sinh 11 round keys đối với AES-128. Các thao tác lỗi gồm RotWord, SubWord và XOR Rcon.

Thành phần	Mô tả
RotWord	Quay [a0,a1,a2,a3] thành [a1,a2,a3,a0]
SubWord	Áp S-box lên từng byte của word
Rcon	Hàng số vòng: 01,02,04,08,10,20,40,80,1B,36
XOR chain	Word mới phụ thuộc word trước và word cách 4 vị trí

Khi bắt gặp một buffer 176 byte được tạo từ key 16 byte, rất có thể đó là expanded key AES-128 (11×16 byte). Với AES-192/AES-256, kích thước và quy tắc expansion khác.

Tìm key đúng ở đâu?

- Nếu breakpoint nằm trong round, pointer có thể trở vào expanded key chứ không phải original key.
- Đi ngược caller đến nơi key schedule được gọi; argument đầu vào thường là original key.
- Nếu dùng BCrypt, kiểm tra buffer key object không đồng nghĩa với raw key; log tại BCryptGenerateSymmetricKey hoặc trước đó.

2.9. CBC, CTR và GCM dưới góc nhìn data flow

CBC encryption XOR plaintext block với ciphertext trước đó rồi mới AES-encrypt. CBC decryption AES-decrypt trước, sau đó XOR ciphertext trước. Block đầu dùng IV.

$$C_i = E_k(P_i \oplus C_{i-1}), \text{ với } C_{-1} = IV$$

$$P_i = D_k(C_i) \oplus C_{i-1}$$

CTR biến block cipher thành stream cipher: mã hóa nonce||counter để tạo keystream rồi XOR dữ liệu. Encrypt và decrypt dùng cùng thao tác XOR.

$$C_i = P_i \oplus E_k(\text{nonce} || \text{counter}_i)$$

Quan sát trong code	Mode nghi ngờ
Decrypt block rồi XOR block ciphertext trước	CBC decrypt
Encrypt counter tăng dần rồi XOR input	CTR
Không chaining, mỗi block gọi độc lập	ECB
CTR-like + phép nhân trường cho authentication	GCM

GCM trả thêm authentication tag. Nếu chỉ giải được plaintext nhưng bỏ qua kiểm tra tag, bạn chưa mô phỏng đầy đủ hành vi của chương trình.

2.10. Padding: PKCS#7 và lỗi thường gặp

Với block size 16, PKCS#7 luôn thêm padding. Nếu thiếu 5 byte thì thêm năm byte 0x05. Nếu plaintext đã vừa block, thêm nguyên một block gồm 16 byte 0x10.

Độ dài thiếu	Bytes padding
1	01
2	02 02
5	05 05 05 05 05
16	10 lặp 16 lần

```
def pkcs7_unpad(data: bytes, block=16) -> bytes:
    if not data or len(data) % block:
        raise ValueError("invalid length")
    n = data[-1]
    if n == 0 or n > block or data[-n:] != bytes([n]) * n:
        raise ValueError("invalid padding")
    return data[:-n]
```

Đừng cắt bằng `rstrip`

`data.rstrip(b'\x10')` không phải PKCS#7 unpad và có thể xóa dữ liệu thật. Phải kiểm tra giá trị byte cuối và toàn bộ chuỗi padding.

2.11. ChaCha20: nhận diện ARX cipher

ChaCha20 dùng Add-Rotate-XOR (ARX) trên state 16 word 32-bit. Dấu hiệu nổi bật là chuỗi rotate cố định 16,12,8,7 và constants ASCII expand 32-byte k.

```
def quarter_round(s, a, b, c, d):  
    s[a] = (s[a] + s[b]) & 0xffffffff; s[d] ^= s[a]; s[d] = rol32(s[d], 16)  
    s[c] = (s[c] + s[d]) & 0xffffffff; s[b] ^= s[c]; s[b] = rol32(s[b], 12)  
    s[a] = (s[a] + s[b]) & 0xffffffff; s[d] ^= s[a]; s[d] = rol32(s[d], 8)  
    s[c] = (s[c] + s[d]) & 0xffffffff; s[b] ^= s[c]; s[b] = rol32(s[b], 7)
```

State gồm 4 constant words, 8 key words, counter và nonce. Sau 20 round, state kết quả được cộng với state ban đầu và serialize little-endian để tạo 64 byte keystream.

Phân biệt với custom ARX

Hãy tìm đồng thời constants ASCII, state 16 words, 20 rounds và rotate 16/12/8/7. Chỉ thấy add-xor-rotate là chưa đủ kết luận ChaCha20.

PHẦN 3

HASH, CHECKSUM & API HASHING

Phân biệt phép biến đổi một chiều với mã hóa và nhận diện chúng qua state, padding và hằng số.

3.1. Magic constants

Thuật toán	Magic constants / đặc trưng
MD5	67452301 EFCDA8B9 98BADCFE 10325476; rotate 7/12/17/22...
SHA-1	67452301 EFCDA8B9 98BADCFE 10325476 C3D2E1F0; 80 round
SHA-256	6A09E667 BB67AE85 3C6EF372 A54FF53A ...; 64 round constants
CRC32	EDB88320 hoặc bảng 256 DWORD; xử lý từng byte
FNV-1a 32	Offset 811C9DC5; prime 01000193; XOR rồi nhân

Magic constants là điểm vào tốt, nhưng không phải bằng chứng tuyệt đối. Một binary có thể chứa constants vì link cả thư viện, trong khi hàm đó không bao giờ được gọi. Hãy xác nhận bằng cross-reference và call graph.

3.2. Hash khác encryption như thế nào?

Tiêu chí	Hash	Encryption
Mục đích	Fingerprint / kiểm tra toàn vẹn	Bảo mật nội dung
Đảo ngược	Không có phép decrypt chung	Có decrypt nếu có key
Độ dài output	Cố định	Phụ thuộc dữ liệu và mode
Key	Không bắt buộc; HMAC có key	Có key
Kiểm tra password	So sánh digest	Không nên mã hóa password để kiểm tra

Trong crackme

Nếu chương trình hash input rồi so sánh digest, không thể 'decrypt hash'. Hướng giải là tìm input qua constraint, dictionary/brute force có giới hạn, side channel, hoặc bypass nhánh kiểm tra trong phạm vi bài tập.

3.3. API hashing trong malware

Malware có thể tránh lưu chuỗi như VirtualAlloc bằng cách duyệt export table của DLL, băm từng tên hàm rồi so sánh với hằng số đã nhúng. Kết quả là import table nghèo nàn nhưng runtime vẫn resolve được API.

Quy trình nhận diện:

1. Tìm code đi qua PEB/Ldr hoặc parse PE export directory.
2. Xác định vòng lặp hash ký tự: hướng duyệt, chữ hoa/thường, terminator, seed và độ rộng word.
3. Viết lại đúng thuật toán; băm danh sách export của DLL nghi vấn.
4. Tạo ánh xạ hash → tên API và gắn lại tên trong IDA/Ghidra.

```
def fnv1a32(name: bytes) -> int:
    h = 0x811C9DC5
    for ch in name:
        h ^= ch
        h = (h * 0x01000193) & 0xffffffff
    return h

for api in (b"VirtualAlloc", b"CreateProcessW", b"WriteProcessMemory"):
    print(api.decode(), hex(fnv1a32(api)))
```

Không đoán biến thể

DJB2, ROR13, CRC32 và FNV thường bị sửa seed, rotate count hoặc chuẩn hóa chữ. Lấy chính xác pseudocode từ mẫu rồi mô phỏng; đừng dùng script có sẵn chỉ vì tên thuật toán trông giống nhau.

3.4. Cấu trúc chung của hash theo block

MD5, SHA-1 và SHA-256 đều xử lý message theo block 512 bit. Trước khi nén, message được padding: thêm bit 1, thêm đủ bit 0 và cuối cùng thêm độ dài message. Endianness của trường độ dài và message schedule khác nhau giữa thuật toán.

Thuật toán	Block	Digest	Endian nổi bật	Số vòng
MD5	64 byte	16 byte	Little-endian	64
SHA-1	64 byte	20 byte	Big-endian	80
SHA-256	64 byte	32 byte	Big-endian	64

Trong pseudocode, hãy tách ba tầng: hàm update gom dữ liệu vào buffer; hàm compression xử lý block; hàm final thêm padding và xuất digest. Magic constants thường nằm trong compression/init, còn padding lộ rõ ở final.

Ví dụ padding SHA-256 cho 'abc'

Ba byte 61 62 63 được theo sau bởi 80, sau đó là 52 byte 00 và trường độ dài 64-bit big-endian bằng 24 bit: 00 00 00 00 00 00 00 18. Tổng cộng đúng một block 64 byte.

Nhận diện update loop

Nếu hàm nhận pointer+length, copy vào buffer 64 byte và gọi compression mỗi khi đầy, đó thường là hash context update chứ chưa phải compression function.

3.5. SHA-256: message schedule và compression

Mỗi block tạo 64 word W . $W[0..15]$ đọc trực tiếp từ block theo big-endian; $W[16..63]$ được mở rộng bằng rotate/shift. Compression duy trì tám state words $a..h$.

$$W[t] = \sigma_1(W[t-2]) + W[t-7] + \sigma_0(W[t-15]) + W[t-16]$$

```
def rotr(x, n):  
    return ((x >> n) | (x << (32-n))) & 0xffffffff  
  
def small_sigma0(x): return rotr(x, 7) ^ rotr(x, 18) ^ (x >> 3)  
def small_sigma1(x): return rotr(x, 17) ^ rotr(x, 19) ^ (x >> 10)  
def choose(x, y, z): return (x & y) ^ (~x & z)  
def majority(x, y, z): return (x & y) ^ (x & z) ^ (y & z)
```

Dấu hiệu trong assembly là nhiều rotate với counts 2,6,7,11,13,17,18,19,22,25 và bảng 64 constants bắt đầu bằng 0x428A2F98. Trên CPU mới, compiler hoặc thư viện có thể dùng SHA extensions nên hình dạng sẽ khác.

3.6. CRC32: checksum có thể đảo theo ngữ cảnh

CRC không được thiết kế chống giả mạo. Nó phát hiện lỗi truyền dẫn tuyến tính và thường dùng polynomial 0x04C11DB7 hoặc dạng reflected 0xEDB88320. Table-driven CRC xử lý mỗi byte bằng lookup 256 DWORD.

```
def crc32_reflected(data: bytes, init=0xffffffff):
    crc = init
    for b in data:
        crc ^= b
        for _ in range(8):
            crc = (crc >> 1) ^ (0xEDB88320 if crc & 1 else 0)
    return crc ^ 0xffffffff
```

Các tham số CRC phải ghi đủ: width, polynomial, init, refin, refout và xorout. Hai hàm đều được gọi là CRC32 nhưng khác init/xorout có thể cho kết quả khác hoàn toàn.

Dấu hiệu	CRC	Cryptographic hash
Mục tiêu	Phát hiện lỗi ngẫu nhiên	Chống collision/preimage theo thiết kế
Cấu trúc	Tuyến tính trên GF(2)	Phi tuyến, nhiều round
Tốc độ	Rất nhanh	Chậm hơn nhưng vẫn tối ưu
Dùng để xác thực	Không an toàn nếu attacker sửa dữ liệu	Cần HMAC/signature nếu cần xác thực nguồn

3.7. API hashing: dựng lại resolver

Một resolver điển hình đọc PEB để tìm base của kernel32/ntdll, parse DOS header và NT headers, lấy Export Directory, duyệt AddressOfNames rồi bấm từng chuỗi. Khi hash khớp, ordinal ánh xạ tới AddressOfFunctions.

1. Đọc e_lfanew tại DOS header để tới NT headers.
2. Lấy RVA Export Directory từ Optional Header DataDirectory.
3. Duyệt NumberOfNames và mảng name RVAs.
4. Hash tên API; khi khớp, lấy ordinal rồi function RVA.
5. Kiểm tra forwarded export; RVA có thể trỏ tới chuỗi DLL.Func thay vì code.

```
# Khung brute-force tên export sau khi đã trích danh sách tên
targets = {0x12345678, 0x9abcdef0}
for name in exported_names:
    h = malware_hash(name.encode('ascii'))
    if h in targets:
        print(hex(h), name)
```

Forwarded export

Nếu function RVA nằm trong vùng export directory và trỏ tới chuỗi như `KERNELBASE.VirtualAlloc`, resolver phải xử lý forwarder. Bỏ qua bước này có thể khiến địa chỉ kết quả không phải executable code.

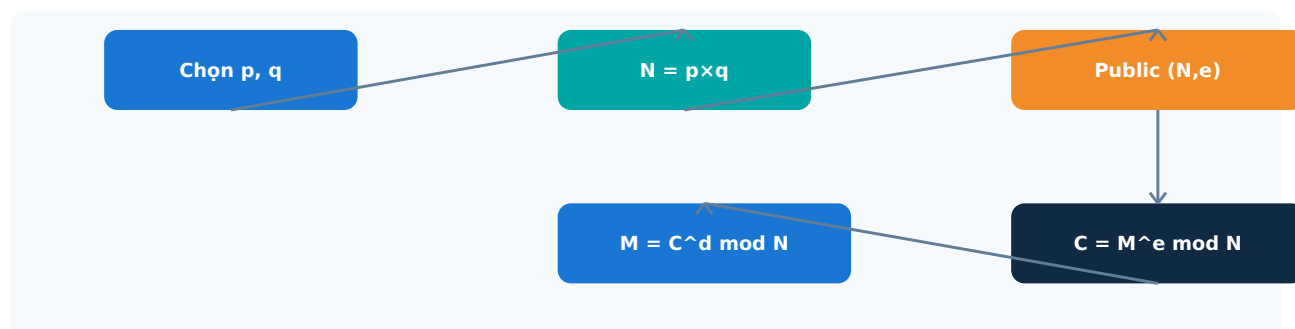
PHẦN 4

MẬT MÃ KHÓA CÔNG KHAI

RSA, ECC và các lỗi triển khai thường xuất hiện trong CTF hoặc phân tích mã độc.

4.1. RSA

RSA dựa trên phép lũy thừa modulo. Public key là (N, e) , private key là d . Với cách trình bày rút gọn: ciphertext $C = M^e \bmod N$ và plaintext $M = C^d \bmod N$.



Các lỗi/điểm yếu thường gặp trong bài tập:

- N quá nhỏ: có thể phân tích thừa số bằng công cụ phù hợp hoặc cơ sở dữ liệu factor.
- p và q quá gần: Fermat factorization hội tụ nhanh.
- Private exponent d quá nhỏ: có thể phù hợp với Wiener attack.
- Cùng modulus hoặc prime bị dùng lại giữa nhiều key: GCD giữa các modulus có thể lộ prime chung.
- Textbook RSA không padding: có nhiều tấn công đại số; triển khai thực tế cần OAEP/PSS đúng chuẩn.

Góc nhìn RE

Trong binary, thường giá trị cần tìm không phải thuật toán RSA mà là vị trí public key, định dạng PEM/DER, hàm import key và buffer trước/sau lời gọi crypto API.

4.2. ECC và ECDSA

ECC đạt mức an toàn cao với khóa ngắn hơn RSA. Trong ECDSA, chữ ký gồm (r, s) và phụ thuộc vào private key d cùng nonce bí mật k . Nếu k bị lộ hoặc tái sử dụng, private key có thể bị khôi phục.

Nếu hai chữ ký dùng cùng k thì $r_1 = r_2$

$$k \equiv (z_1 - z_2) \cdot (s_1 - s_2)^{-1} \pmod{n}$$

$$d \equiv (s_1 \cdot k - z_1) \cdot r^{-1} \pmod{n}$$

Trong đó z là integer rút ra từ hash của thông điệp, n là order của base point, và x^{-1} là nghịch đảo modulo n .

```
# Công thức minh họa; Python >= 3.8 hỗ trợ pow(x, -1, n)
k = ((z1 - z2) * pow((s1 - s2) % n, -1, n)) % n
d = ((s1 * k - z1) * pow(r, -1, n)) % n
```

Điều kiện bắt buộc

Hai chữ ký phải thuộc cùng curve/key và thực sự dùng cùng nonce (thường thể hiện qua cùng r). Mọi phép toán phải lấy modulo n ; nhầm modulo p của trường là lỗi rất phổ biến.

4.3. Dấu vết khóa công khai trong binary

- Chuỗi PEM: BEGIN PUBLIC KEY, BEGIN RSA PUBLIC KEY, BEGIN CERTIFICATE.
- DER/ASN.1: thường bắt đầu bằng 0x30 và có cấu trúc TLV.
- CryptoAPI/CNG: CryptImportKey, CryptDecodeObjectEx, BCryptImportKeyPair, NCryptImportKey.
- OpenSSL: d2i_PUBKEY, PEM_read_bio_PUBKEY, EVP_PKEY, RSA_public_decrypt...

4.4. Số học modulo: nền bắt buộc trước RSA/ECC

Hai số a và b đồng dư modulo n nếu chúng có cùng phần dư khi chia cho n . Viết $a \equiv b \pmod{n}$. Trong crypto, mọi phép cộng, trừ, nhân và lũy thừa đều được thu về miền hữu hạn.

$$a \equiv b \pmod{n} \Leftrightarrow n \text{ chia hết } (a - b)$$

Ví dụ $17 \equiv 5 \pmod{12}$ vì $17 - 5 = 12$. Phép modulo không phải phép lấy phần dư máy móc; nó tạo ra các lớp tương đương và cho phép định nghĩa nghịch đảo.

GCD và Euclid mở rộng

Nghịch đảo $a^{-1} \pmod{n}$ tồn tại khi $\gcd(a, n) = 1$. Euclid mở rộng tìm x, y sao cho $ax + ny = \gcd(a, n)$. Nếu $\gcd = 1$ thì x chính là nghịch đảo modulo của a .

```
def egcd(a, b):
    if b == 0:
        return a, 1, 0
    g, x1, y1 = egcd(b, a % b)
    return g, y1, x1 - (a // b) * y1

def invmod(a, n):
    g, x, _ = egcd(a, n)
    if g != 1:
        raise ValueError("inverse does not exist")
    return x % n
```

Ví dụ: nghịch đảo của 7 modulo 40 là 23 vì $7 \times 23 = 161 \equiv 1 \pmod{40}$. Đây chính là phép dùng để tính d từ e trong RSA và tính k^{-1} trong ECDSA.

4.5. Lũy thừa modulo nhanh

Tính M^e trực tiếp là bất khả thi khi e có hàng trăm bit. Square-and-multiply tính lũy thừa trong $O(\log e)$ phép nhân modulo. Python cung cấp `pow(base, exponent, modulus)`.

```
def modexp(base, exp, mod):
    result = 1
    base %= mod
    while exp:
        if exp & 1:
            result = (result * base) % mod
        base = (base * base) % mod
        exp >>= 1
    return result

assert modexp(7, 13, 33) == pow(7, 13, 33)
```

Trong assembly, big-integer modular exponentiation không còn là một lệnh đơn. Bạn sẽ thấy nhiều hàm xử lý limb arrays, multiplication, reduction và allocation. Vì vậy trong binary thật, imports và object types của thư viện thường hữu ích hơn cố nhận dạng từ từng phép nhân.

Montgomery reduction

Thư viện crypto thường dùng Montgomery multiplication để giảm chi phí modulo lớn. Constants như R , $R^2 \bmod N$ và n' có thể xuất hiện trong context. Nếu thấy arithmetic trên mảng word cùng vòng carry dài, đừng nhầm đó là custom cipher; có thể chỉ là big-number backend.

4.6. RSA key generation: ví dụ số nhỏ từng bước

Ví dụ dưới đây dùng số cực nhỏ chỉ để hiểu toán, tuyệt đối không an toàn.

Bước	Tính toán	Kết quả
1	Chọn $p=61$, $q=53$	Hai prime
2	$N=pq$	3233
3	$\phi(N)=(p-1)(q-1)$	3120
4	Chọn e sao cho $\gcd(e,\phi)=1$	$e=17$
5	$d=e^{-1} \bmod \phi(N)$	$d=2753$
6	Public key	$(N=3233, e=17)$
7	Private key	$d=2753$; thực tế còn lưu $p, q, dp, dq, qInv$

Mã hóa message integer $M=65$:

$$C = 65^{17} \bmod 3233 = 2790$$

Giải mã:

$$M = 2790^{2753} \bmod 3233 = 65$$

```
p, q = 61, 53
n = p*q
phi = (p-1)*(q-1)
e = 17
d = pow(e, -1, phi)
m = 65
c = pow(m, e, n)
assert pow(c, d, n) == m
print(n, phi, d, c)
```

Điểm quan trọng

RSA không mã hóa chuỗi trực tiếp bằng công thức trên trong triển khai an toàn. Dữ liệu phải được encode và padding theo chuẩn như OAEP; chữ ký dùng PSS hoặc scheme chuẩn tương ứng.

4.7. Vì sao RSA giải mã được?

Ta chọn d sao cho $ed \equiv 1 \pmod{\phi(N)}$, nghĩa là $ed=1+k\phi(N)$. Theo định lý Euler, với M nguyên tố cùng nhau với N : $M^{\phi(N)} \equiv 1 \pmod{N}$. Vì thế:

$$M^{ed} = M^{1+k\phi(N)} = M \cdot (M^{\phi(N)})^k \equiv M \pmod{N}$$

Chứng minh đầy đủ cho mọi M thường dùng CRT và xét riêng modulo p, q . Điều cần nhớ khi reverse: e, d, N không phải ba số ngẫu nhiên; chúng thỏa quan hệ toán học với p, q .

Carmichael $\lambda(N)$

Một số triển khai dùng $\lambda(N)=\text{lcm}(p-1, q-1)$ thay cho $\phi(N)$. Khi đó $d=e^{-1} \pmod{\lambda(N)}$. Cả hai đều dẫn đến key hợp lệ nếu điều kiện được đáp ứng. Vì vậy script kiểm tra key không nên mặc định chỉ có một biểu diễn d .

RSA CRT optimization

Private operation thường được tăng tốc bằng $dp=d \pmod{p-1}$, $dq=d \pmod{q-1}$, $qInv=q^{-1} \pmod{p}$. Nếu binary chứa $p, q, dp, dq, qInv$, nó có thể thực hiện CRT thay vì một modexp lớn.

Thành phần	Ý nghĩa
dp	$d \pmod{p-1}$
dq	$d \pmod{q-1}$
$qInv$	$q^{-1} \pmod{p}$
$m1$	$c^{dp} \pmod{p}$
$m2$	$c^{dq} \pmod{q}$

4.8. Biểu diễn message và giới hạn kích thước

RSA thao tác trên integer $0 \leq M < N$. Byte string được chuyển thành integer theo big-endian trong hầu hết chuẩn. Với modulus k byte, plaintext sau encoding phải nhỏ hơn N ; do padding, kích thước hữu dụng nhỏ hơn k .

```
msg = b"HELLO"
m = int.from_bytes(msg, "big")
roundtrip = m.to_bytes((m.bit_length()+7)//8, "big")
assert roundtrip == msg
```

Với RSA-OAEP dùng SHA-256 và modulus k byte, message tối đa có độ dài $k - 2 \cdot 32 - 2$. Ví dụ RSA-2048 có $k=256$ byte, nên tối đa 190 byte. Đây là lý do hệ thống thực tế dùng hybrid encryption: RSA mã hóa một khóa AES ngẫu nhiên, còn AES mã hóa dữ liệu lớn.

Bước hybrid encryption	Dữ liệu
1	Sinh khóa AES và nonce ngẫu nhiên
2	AES-GCM mã hóa file/config/payload
3	RSA-OAEP mã hóa khóa AES
4	Lưu encrypted_key nonce ciphertext tag

Góc nhìn RE

Nếu malware dùng RSA, private key thường không nằm trong mẫu. Mục tiêu phân tích thường là tìm session key trước khi bị RSA-wrap, hoặc hiểu định dạng blob gửi về server.

4.9. RSA key formats: PEM, DER và ASN.1

PEM là DER được Base64 và bọc bằng header/footer. DER là encoding nhị phân của ASN.1. Một public key có thể xuất hiện dưới dạng PKCS#1 RSAPublicKey hoặc SubjectPublicKeyInfo; hai cấu trúc không giống nhau dù chứa cùng N,e.

Header PEM	Nội dung thường gặp
BEGIN PUBLIC KEY	SubjectPublicKeyInfo; chứa algorithm identifier + BIT STRING
BEGIN RSA PUBLIC KEY	PKCS#1; trực tiếp chứa modulus và exponent
BEGIN PRIVATE KEY	PKCS#8; wrapper thuật toán cho private key
BEGIN RSA PRIVATE KEY	PKCS#1 RSA private key gồm n,e,d,p,q,...
BEGIN CERTIFICATE	X.509 certificate; public key chỉ là một phần

DER dùng TLV: tag, length, value. Tag 0x30 thường là SEQUENCE, 0x02 là INTEGER. Modulus dương có thể có byte 00 đứng đầu để tránh bị hiểu là số âm.

```
# Dùng thư viện thay vì tự parse ASN.1 trong công việc thật
from cryptography.hazmat.primitives import serialization

pub = serialization.load_pem_public_key(pem_bytes)
numbers = pub.public_numbers()
print(hex(numbers.n), numbers.e)
```

4.10. Tấn công 1 - Factor N nhỏ

Nếu $N=pq$ đủ nhỏ, factorization trực tiếp khôi phục p, q . Sau đó tính $\phi(N)$ hoặc $\lambda(N)$, suy ra d và giải mã. CTF thường cố tình dùng modulus 256-512 bit hoặc prime yếu; RSA hiện đại dùng ít nhất 2048 bit với prime sinh đúng cách.

```
# Sau khi đã factor được n thành p, q
phi = (p - 1) * (q - 1)
d = pow(e, -1, phi)
m = pow(c, d, n)
pt = m.to_bytes((m.bit_length()+7)//8, 'big')
print(pt)
```

Khi dùng công cụ factor, phải xác minh $p \times q = N$ và p, q thực sự prime. Nếu N có nhiều hơn hai factor hoặc prime power, công thức ϕ cần điều chỉnh.

Độ lớn không phải lỗi duy nhất

Một modulus 2048-bit vẫn yếu nếu hai mẫu chia sẻ prime. Vì vậy kiểm tra GCD hàng loạt giữa nhiều modulus là kỹ thuật thực dụng trong phân tích tập key/certificate.

4.11. Tấn công 2 - Fermat khi p và q gần nhau

Nếu p và q gần nhau, N có thể viết $N=a^2-b^2=(a-b)(a+b)$. Bắt đầu $a=\text{ceil}(\text{sqrt}(N))$ và tăng a cho đến khi a^2-N là số chính phương.

```
from math import isqrt

def fermat_factor(n):
    a = isqrt(n)
    if a*a < n:
        a += 1
    while True:
        b2 = a*a - n
        b = isqrt(b2)
        if b*b == b2:
            return a-b, a+b
        a += 1
```

Thuật toán nhanh khi $|p-q|$ nhỏ, nhưng vô dụng với prime ngẫu nhiên cách xa đúng chuẩn. Dấu hiệu CTF: đề nhấn mạnh prime liên tiếp, timestamp gần nhau hoặc generator tạo q từ p.

Xác minh

- Kiểm tra $p \times q = N$.
- Kiểm tra $\text{gcd}(e, (p-1)(q-1)) = 1$.
- Tạo d và giải mã; sau đó kiểm tra padding/format chứ không chỉ printable text.

4.12. Tấn công 3 - Shared prime và batch GCD

Nếu $N_1=pq_1$ và $N_2=pq_2$, thì $\gcd(N_1, N_2)=p$. Chỉ một lần GCD đã factor cả hai modulus. Lỗi này xảy ra khi RNG yếu sinh lại một prime trong quá trình tạo key.

```
from math import gcd

g = gcd(n1, n2)
if 1 < g < n1:
    p = g
    q1 = n1 // p
    q2 = n2 // p
    print('shared prime', p)
```

Với hàng nghìn modulus, so sánh mọi cặp là $O(n^2)$; có thuật toán batch GCD dựa trên product tree/remainder tree. Trong bài CTF nhỏ, vòng lặp cặp thường đủ.

Không nhầm cùng modulus

Hai key có cùng N nhưng exponent khác là bài common modulus. Hai modulus khác nhau nhưng $\gcd > 1$ là shared-prime. Công thức khai thác hoàn toàn khác.

4.13. Tấn công 4 - Common modulus

Cùng plaintext M được mã hóa dưới cùng N nhưng hai exponent e_1, e_2 nguyên tố cùng nhau. Euclid mở rộng tìm a, b sao cho $ae_1 + be_2 = 1$. Khi đó $M = C_1^a \cdot C_2^b \bmod N$.

Nếu hệ số âm, cần dùng modular inverse của ciphertext. Attack không cần factor N nhưng yêu cầu cùng message, cùng modulus và $\gcd(e_1, e_2) = 1$.

```
g, a, b = egcd(e1, e2)
assert g == 1

def signed_pow(c, k, n):
    return pow(c, k, n) if k >= 0 else pow(pow(c, -1, n), -k, n)

m = (signed_pow(c1, a, n) * signed_pow(c2, b, n)) % n
```

Nếu message đã được OAEP với randomness khác nhau, hai integer encoded message không còn giống nhau nên common modulus attack theo dạng đơn giản này không áp dụng.

4.14. Tấn công 5 - Low exponent và Håstad broadcast

Nếu $e=3$ và $M^3 < N$, phép modulo chưa thực sự cuộn; ciphertext chỉ là M^3 và có thể lấy căn bậc ba nguyên. Padding chuẩn ngăn điều kiện nguy hiểm này.

```
# Cần integer nth-root, không dùng float
from sympy import integer_nthroot
m, exact = integer_nthroot(c, 3)
if exact:
    print(m.to_bytes((m.bit_length()+7)//8, 'big'))
```

Håstad broadcast xảy ra khi cùng M được gửi cho ít nhất e recipient với các modulus đôi một nguyên tố cùng nhau và exponent nhỏ e . Chinese Remainder Theorem ghép các ciphertext để thu M^e dưới dạng integer, sau đó lấy căn bậc e .

Điều kiện phải kiểm tra

- Cùng encoded message M , không phải chỉ cùng plaintext trước randomized padding.
- Các modulus đủ độc lập để CRT áp dụng.
- Có ít nhất e ciphertext khi exponent bằng e .

4.15. Tấn công 6 - Wiener khi d quá nhỏ

Nếu d rất nhỏ so với N , phân số e/N có thể tiết lộ k/d qua continued fractions. Các convergent p_i/q_i cung cấp ứng viên k,d ; mỗi ứng viên được kiểm tra bằng quan hệ $ed-1$ chia hết cho k và phương trình tìm p,q có nghiệm nguyên.

Bước	Thao tác kiểm tra
1	Tạo continued fraction của e/N
2	Sinh các convergent k/d
3	Nếu $k=0$ hoặc $(ed-1) \bmod k \neq 0$ thì bỏ
4	$\phi=(ed-1)/k$; suy ra $S=N-\phi+1=p+q$
5	Kiểm tra discriminant S^2-4N là số chính phương

Điều kiện kinh điển xấp xỉ $d < N^{0.25} / 3$ cùng một số giả định về p,q . Không phải cứ d 'trông nhỏ' là attack chắc chắn hoạt động.

Góc nhìn RE

Nếu binary nhúng private key đầy đủ, không cần Wiener. Attack hữu ích khi chỉ có public (N,e) và challenge cố ý sinh d yếu.

4.16. OAEP và PSS: padding không phải phần phụ

Textbook RSA mang tính xác định: cùng M cho cùng C. OAEP dùng hash, mask generation function và seed ngẫu nhiên để tạo encoding xác suất cho encryption. PSS làm tương tự cho signature, đưa randomness vào encoding chữ ký.

Scheme	Dùng cho	Mục tiêu
OAEP	RSA encryption	Randomized encoding, chống nhiễu tấn công trên textbook RSA
PKCS#1 v1.5 encryption	Legacy encryption	Có cấu trúc padding; cần xử lý oracle cực cẩn thận
PSS	RSA signature	Signature encoding hiện đại, có salt
PKCS#1 v1.5 signature	Legacy signature	DigestInfo có cấu trúc xác định

Trong reverse, xác định hash algorithm, MGF hash, label, salt length và padding constant. Chỉ biết RSA-2048 chưa đủ để tái tạo kết quả.

API clue

- BCryptEncrypt/Decrypt nhận padding info và flag BCRYPT_PAD_OAEP hoặc BCRYPT_PAD_PKCS1.
- OpenSSL EVP_PKEY_CTX cấu hình rsa_padding_mode và OAEP digest.
- Java Cipher transformation có thể ghi rõ RSA/ECB/OAEPWithSHA-256AndMGF1Padding.

4.17. RSA trong IDA/x64dbg: workflow thực dụng

1. Triage strings/imports: PEM header, BCrypt, Crypt32, OpenSSL, mbedTLS, wolfSSL.
2. Tìm nơi key được parse/import. Đặt breakpoint sau khi parse thành công.
3. Tại operation call, log input length, output length, padding flags và key handle/context.
4. Nếu mục tiêu là plaintext, breakpoint sau decrypt thường dễ hơn reverse big-number code.
5. Nếu mục tiêu là protocol, tái dựng chính xác serialization của encrypted blob.

Các câu hỏi phải trả lời

- Đây là public operation hay private operation? Encrypt, decrypt, sign hay verify?
- Key ở dạng raw DER/PEM hay đã thành opaque handle?
- Padding nào? Hash nào? Message có bị hash trước không?
- Byte order của integer và layout output ra sao?
- Có hybrid encryption không; session key xuất hiện ở đâu trước khi wrap?

Không reverse quá sâu

Nếu binary gọi thư viện chuẩn, biên API là nơi giàu thông tin nhất. Chỉ bước vào big-number implementation khi API bị inline, statically linked hoặc đã bị tùy biến.

4.18. ECC: trường hữu hạn và điểm trên đường cong

Một đường cong Weierstrass thường có dạng $y^2 \equiv x^3 + ax + b \pmod{p}$. Điểm G có order n; private key d là integer, public key $Q = dG$. Phép nhân ở đây là scalar multiplication - cộng điểm G với chính nó d lần theo thuật toán hiệu quả.

$$Q = d \cdot G$$

Point addition dùng slope λ và phép nghịch đảo modulo p. Khi $P \neq Q$:

$$\lambda = (y_2 - y_1)(x_2 - x_1)^{-1} \pmod{p}$$

$$x_3 = \lambda^2 - x_1 - x_2 \pmod{p}; y_3 = \lambda(x_1 - x_3) - y_1 \pmod{p}$$

Point doubling dùng $\lambda = (3x_1^2 + a)(2y_1)^{-1} \pmod{p}$. Điểm vô cực đóng vai trò phần tử đơn vị.

Trong thực tế không tự viết ECC cho sản phẩm. Ở góc nhìn RE, công thức giúp nhận diện arithmetic và hiểu key/signature, không phải khuyến khích triển khai crypto thủ công.

4.19. ECDSA ký và xác minh

1. Băm message thành z .
2. Chọn nonce bí mật k trong $[1, n-1]$.
3. Tính $R=kG$ và $r=R.x \bmod n$; nếu $r=0$ chọn k khác.
4. Tính $s=k^{-1}(z+r \cdot d) \bmod n$; nếu $s=0$ chọn k khác.
5. Signature là (r,s) , thường DER-encode thành SEQUENCE hai INTEGER.

Xác minh dùng public key Q , tính $w=s^{-1}$, $u_1=zw$, $u_2=rw$ và điểm $X=u_1G+u_2Q$. Chữ ký hợp lệ nếu $X.x \bmod n = r$.

Giá trị	Bí mật?	Vai trò
d	Có	Private signing key
k	Có và dùng một lần	Nonce cho mỗi chữ ký
$Q=dG$	Không	Public verification key
r,s	Không	Signature
z	Không	Hash message đã chuẩn hóa

4.20. ECDSA nonce reuse: suy ra công thức

Hai chữ ký dùng cùng k có cùng điểm R và do đó cùng r . Ta có:

$$s_1 = k^{-1}(z_1 + r \cdot d) \bmod n$$

$$s_2 = k^{-1}(z_2 + r \cdot d) \bmod n$$

Lấy hai phương trình trừ nhau: $s_1 - s_2 = k^{-1}(z_1 - z_2)$. Nhân hai vế với k và nghịch đảo $(s_1 - s_2)$:

$$k = (z_1 - z_2)(s_1 - s_2)^{-1} \bmod n$$

Sau khi có k , từ $s_1 k = z_1 + r \cdot d$ suy ra:

$$d = (s_1 k - z_1) r^{-1} \bmod n$$

```
def recover_ecdsa_key(r, s1, s2, z1, z2, n):
    k = ((z1 - z2) * pow((s1 - s2) % n, -1, n)) % n
    d = ((s1 * k - z1) * pow(r, -1, n)) % n
    return k, d
```

Sau khi khôi phục d , phải xác minh dG bằng public key Q và có thể ký test message để kiểm tra. Nếu r giống nhau nhưng signature dùng key/curve khác, công thức không hợp lệ.

Các biến thể

Nonce bị bias hoặc lộ một phần bit có thể dẫn tới lattice attack, nhưng phức tạp hơn reuse hoàn toàn. Deterministic ECDSA (RFC 6979) tạo k từ key+message để tránh RNG hỏng.

4.21. Encoding public key và signature ECC

Public key uncompressed thường là $0x04 || X || Y$. Compressed form dùng $0x02$ hoặc $0x03$ tùy parity của Y, sau đó X. Với curve 256-bit, X và Y mỗi phần thường 32 byte.

Dạng	Prefix	Kích thước secp256k1/P-256
Uncompressed public key	04	65 byte
Compressed public key	02 hoặc 03	33 byte
Raw signature	Không cố định prefix	$r s$ thường 64 byte
DER signature	30 SEQUENCE	Biến đổi, thường ~70-72 byte

DER INTEGER có thể thêm byte 00 khi bit cao bằng 1, nên r và s không luôn chiếm đúng 32 byte bên trong DER. Parser không được cắt signature bằng offset cố định.

Góc nhìn RE

- Tìm tên curve/OID, ví dụ prime256v1, secp256k1, Ed25519.
- Xác định signature là raw hay DER trước khi so sánh.
- Phân biệt ECDSA với EdDSA: thuật toán, encoding và nonce derivation khác nhau.

PHẦN 5

QUY TRÌNH REVERSE CRYPTO THỰC CHIẾN

Một workflow có thể lặp lại: phân loại, hình thành giả thuyết, thu thập số runtime và xác minh bằng script.

5.1. Quy trình 5 bước

Bước	Thao tác	Kết quả cần có
1	Triage strings, imports, constants, entropy và call sites	Danh sách hàm/region nghi vấn
2	Phân tích block size, vòng lặp, state và data flow	Giả thuyết thuật toán + độ tin cậy
3	Đặt breakpoint tại biên: trước/sau hàm crypto hoặc API	Input, output, key, IV/nonce, length
4	Viết script tối thiểu mô phỏng đúng độ rộng integer	Output trùng với binary trên test vector
5	Dùng script cho dữ liệu mục tiêu và lưu bằng chứng	Plaintext/IOC/cấu hình + ghi chú tái lập

Bước xác minh là quan trọng nhất. Nếu script và binary cho cùng output trên một input bạn kiểm soát, mô hình của bạn đã có bằng chứng thực nghiệm. Nếu không khớp, hãy kiểm tra lại byte order, signedness, padding, số vòng, thứ tự cập nhật và mode.

5.2. Static analysis

- Tìm magic constants bằng immediate search hoặc plugin FindCrypt.
- Từ constant/table, đi theo xref tới hàm sử dụng thật sự.
- Đổi tên biến theo vai trò tạm thời: input, output, len, state, key_ptr.
- Xác định độ rộng từng toán hạng; ghi chú nơi compiler chèn cast/truncation.
- Vẽ data flow nhỏ thay vì cố hiểu toàn bộ binary.

5.3. Dynamic analysis

- Breakpoint tại hàm wrapper để hiểu trước khi bước vào implementation.
- Theo calling convention để đọc argument; dereference pointer và dump đúng length.
- Đặt hardware breakpoint vào buffer nếu chưa biết hàm nào ghi dữ liệu.
- So sánh memory trước/sau call; lưu dump nguyên bản để script hóa.

An toàn malware

Phân tích động trong VM cô lập, snapshot sạch, mạng mô phỏng khi cần và không đưa mẫu ra môi trường thật. Ghi lại hash mẫu và trạng thái VM để tái lập.

5.4. Công cụ nên dùng đúng chỗ

Công cụ	Vai trò tốt nhất	Giới hạn cần nhớ
FindCrypt	Quét constants/tables	Có false positive; custom crypto có thể không khớp
IDA / Ghidra	Call graph, xref, pseudocode, type	Decompiler có thể sai signedness/type
x64dbg	Breakpoint, memory dump, Windows user-mode	Anti-debug và timing có thể làm lệch hành vi
Frida	Hook API/hàm runtime và log argument	Cần xác định địa chỉ/signature đúng
Z3	Giải constraint bit-vector	Không thay thế việc mô hình hóa đúng thuật toán
Python	Prototype, decrypt, test vector	Phải mô phỏng overflow và endianness

5.5. Nhật ký phân tích tối thiểu

```

Môu / SHA-256:
Hàm nghi vñ / RVA:
Input pointer + length:
Output pointer + length:
Key / key length:
IV hoặc nonce:
Block size / số vòng:
Magic constants:
Giá thuyt:
Test vector tự tạo:
Kt qu script so với binary:
Đi m còn chưa ch c:

```

5.6. Phân biệt transform, encoding, compression và crypto

Trước khi nhận diện thuật toán cụ thể, hãy phân loại hành vi. Việc này ngăn lỗi phổ biến: cố 'decrypt' Base64, cố 'decode' hash, hoặc coi mọi XOR loop là encryption.

Câu hỏi	Nếu có	Nếu không
Có secret key không?	Có thể là encryption/MAC	Encoding, compression, hash hoặc checksum
Output cố định bất kể input length?	Hash/MAC	Encoding/encryption/compression
Có phép inverse rõ ràng?	Encoding/encryption/reversible transform	Hash hoặc lossy transform
Output entropy cao hơn rõ?	Compression/encryption	Có thể chỉ là permutation/substitution
Có authentication tag?	AEAD/MAC	Không chứng minh được tính toàn vẹn

Một pipeline malware điển hình có thể là: Base64 decode → XOR cấu hình → parse JSON; hoặc RSA unwrap key → AES-GCM decrypt payload → decompress → load PE. Hãy dựng pipeline theo thứ tự runtime.

5.7. Case study A - custom byte transform

Pseudocode giả định:

```
for (i = 0; i < len; i++) {  
    x = input[i] ^ key[i & 7];  
    x = rol8(x, 3);  
    output[perm[i]] = x + 0x17;  
}
```

Để đảo, thực hiện phép nghịch đảo theo thứ tự ngược:

1. Đảo permutation: đưa output[perm[i]] về vị trí i.
2. Trừ 0x17 modulo 256.
3. ROR 3 để đảo ROL 3.
4. XOR lại cùng key byte.

```
def reverse_transform(out, key, perm):  
    tmp = bytearray(len(out))  
    for i, dst in enumerate(perm):  
        tmp[i] = out[dst]  
    plain = bytearray()  
    for i, b in enumerate(tmp):  
        b = (b - 0x17) & 0xff  
        b = ((b >> 3) | (b << 5)) & 0xff  
        plain.append(b ^ key[i & 7])  
    return bytes(plain)
```

Quy tắc vàng

Nếu forward là $A \rightarrow B \rightarrow C \rightarrow D$ thì inverse là $D^{-1} \rightarrow C^{-1} \rightarrow B^{-1} \rightarrow A^{-1}$. Rất nhiều script sai dù từng phép nghịch đảo riêng lẻ đều đúng vì đảo sai thứ tự.

5.8. Case study B - trích key/IV tại CryptoAPI

Giả sử binary dùng BCrypt. Static analysis cho thấy BCryptOpenAlgorithmProvider, BCryptSetProperty, BCryptGenerateSymmetricKey và BCryptDecrypt.

1. Xref chuỗi BCRYPT_AES_ALGORITHM hoặc lời gọi OpenAlgorithmProvider để xác định algorithm handle.
2. Quan sát BCryptSetProperty với BCRYPT_CHAINING_MODE để biết CBC/GCM/ECB.
3. Tại BCryptGenerateSymmetricKey, dump pbSecret với cbSecret: đây thường là raw symmetric key.
4. Tại BCryptDecrypt, ghi pbInput/cbInput, pbIV/cbIV, padding info, output buffer và flags.
5. Sau call thành công, dump đúng số byte pcbResult trả về.

Prototype rút gọn cần nhớ

```
NTSTATUS BCryptDecrypt(
    BCRYPT_KEY_HANDLE hKey,
    PCHAR pbInput, ULONG cbInput,
    VOID *pPaddingInfo,
    PCHAR pbIV, ULONG cbIV,
    PCHAR pbOutput, ULONG cbOutput,
    ULONG *pcbResult, ULONG dwFlags);
```

Trên Windows x64, bốn argument đầu ở RCX,RDX,R8,R9; phần còn lại nằm trên stack. Tuy nhiên breakpoint tại entry có thể bị wrapper thay đổi; luôn đối chiếu prototype và calling convention.

5.9. Case study C - nhận diện thuật toán từ constant

Tìm thấy 0x9E3779B9 chưa đủ kết luận TEA. Quy trình củng cố bằng chứng:

1. Xref constant tới tất cả nơi sử dụng; loại data không được gọi.
2. Kiểm tra vòng lặp quanh constant: 32 round hay số khác?
3. Kiểm tra toán hạng: hai word 32-bit hay mảng n word?
4. Tìm shift 4/5, key[0..3], sum và thứ tự update.
5. Lấy test vector runtime và thử TEA/XTEA/XXTEA đúng endian.

Bằng chứng	Độ mạnh
Chỉ có delta	Yếu
Delta + 32 vòng	Trung bình
Delta + 32 vòng + shift 4/5 + 2 word	Mạnh cho TEA
Script khớp output runtime	Xác nhận thực nghiệm

Cách tư duy này áp dụng cho mọi signature: magic constant là điểm bắt đầu điều tra, không phải đáp án.

5.10. Debugger: theo dữ liệu thay vì step từng lệnh

Step từ entrypoint tới crypto thường lãng phí thời gian. Hãy bắt đầu từ dữ liệu quan tâm và tìm nơi nó được đọc/ghi.

Tình huống	Chiến thuật
Biết buffer ciphertext	Hardware breakpoint on access/read để tìm consumer
Biết plaintext xuất hiện sau decrypt	Memory scan rồi breakpoint on write vùng đó
Biết API crypto	Breakpoint tại API/wrapper và log arguments
Biết file/network source	Breakpoint ReadFile/recv rồi theo buffer
Không biết gì	Triage imports, strings, entropy, call graph và runtime trace ngắn

Khi buffer được copy nhiều lần, gắn nhãn từng phiên bản: raw input, decoded, decrypted, decompressed. So sánh length và vài bytes đầu sau mỗi bước để dựng data lineage.

Breakpoint phải ghi gì?

- Địa chỉ instruction và call stack.
- Pointer, length, sample bytes trước call.
- Return value, output length, sample bytes sau call.
- Thread ID nếu chương trình đa luồng.
- Điều kiện tái lập: input, timestamp/seed và state liên quan.

5.11. Script hóa đúng: signedness, overflow và packing

Decompiler có thể hiển thị int nhưng machine instruction chỉ giữ 8/32 bit. Python không overflow, do đó phải mô phỏng rõ ràng.

```
import struct

def u8(x): return x & 0xff
def u16(x): return x & 0xffff
def u32(x): return x & 0xffffffff
def u64(x): return x & 0xffffffffffffffff

def s8(x):
    x &= 0xff
    return x - 0x100 if x & 0x80 else x

word = struct.unpack('<I', raw[0:4])[0]
raw_again = struct.pack('<I', word)
```

Phân biệt shift logic và arithmetic. Trên x86, shr điền 0; sar nhân bản sign bit. Python >> trên số âm là arithmetic shift. Nếu giá trị phải là unsigned 32-bit, mask trước hoặc sau tùy đúng semantics.

Test từng tầng

Đừng viết một script 200 dòng rồi mới chạy. Kiểm tra từng primitive, từng round hoặc từng block bằng state dump từ debugger. Sai lệch đầu tiên cho biết lỗi nằm ở đâu.

5.12. Bài thực hành tự kiểm tra

Bài 1 - XOR

Cho ciphertext hex và biết plaintext bắt đầu bằng MZ. Khôi phục hai byte key đầu, thử các chu kỳ key length 2..16 và xếp hạng kết quả.

Bài 2 - TEA

Compile một chương trình TEA nhỏ không symbol. Trong IDA, tìm delta, xác định encrypt/decrypt, trích key bằng debugger và viết script kiểm tra một block.

Bài 3 - AES API

Viết chương trình dùng BCrypt AES-CBC. Không nhìn source khi debug: xác định mode, key, IV, ciphertext và padding chỉ từ API calls.

Bài 4 - API hashing

Tạo resolver bấm tên export bằng FNV-1a rồi strip symbol. Reverse hash loop, brute-force export kernel32 và gán lại tên API.

Bài 5 - RSA yếu

Sinh RSA toy với p,q gần nhau. Chỉ giữ N,e,c; dùng Fermat factorization, suy ra d và giải plaintext. Sau đó tăng khoảng cách p,q để quan sát số vòng tăng.

Bài 6 - ECDSA nonce reuse

Tạo hai chữ ký trên hai message với cùng k trong môi trường lab. Khôi phục k,d và xác minh public key.

Tiêu chí hoàn thành

Mỗi bài phải có: giả thuyết, bằng chứng static, bằng chứng dynamic, script tái lập, test vector và ghi chú lỗi đã gặp. Chỉ lấy được plaintext chưa đồng nghĩa đã hiểu kỹ thuật.

PHẦN PHỤ LỤC

CHEAT SHEET NHẬN DIỆN

Bảng tra nhanh dùng khi đang đứng trước pseudocode hoặc assembly chưa có tên.

Nếu thấy	Nghi ngờ	Cần kiểm tra thêm
$S[i]=i$ với 256 phần tử + swap	RC4	Có KSA và PRGA riêng không?
0x9E3779B9; shift 4/5	TEA/XTEA	Block 2×32-bit? chọn key thế nào?
63 7C 77 7B...	AES S-box	Có block 16 byte / Rcon / AES rounds?
0xEDB88320 hoặc table 256 DWORD	CRC32	Index = (crc XOR byte) & 0xff?
$0x811C9DC5 \times 0x01000193$	FNV-1a	XOR trước nhân hay sau nhân?
Rotate + add + xor lặp	ARX/hash/custom	Word size, constants, round count
Duyệt export table + hash tên	API hashing	Seed, case folding, terminator
16-byte blocks + IV	AES/CBC/CTR...	Mode, padding, key length
PEM/DER + big integers	RSA/ECC	API import key và operation cụ thể

Checklist trước khi nói ‘đã giải được’

- Script xử lý đúng input thật và ít nhất một test vector kiểm soát được.
- Kết quả khớp byte-for-byte với binary hoặc thư viện tham chiếu.
- Đã khóa đúng integer width, endianness, padding và mode.
- Đã phân biệt key với IV, state, table hoặc ciphertext.
- Đã lưu lại địa chỉ hàm, breakpoint và dữ liệu cần thiết để tái lập.

MẪU HÀM BIT-VECTOR TRONG PYTHON

```

MASK32 = 0xffffffff

def u32(x):
    return x & MASK32

def rol32(x, n):
    n &= 31
    return u32((x << n) | (x >> ((32 - n) & 31)))

def ror32(x, n):
    n &= 31
    return u32((x >> n) | (x << ((32 - n) & 31)))

def words_le(buf):
    return [int.from_bytes(buf[i:i+4], "little")
            for i in range(0, len(buf), 4)]

```

Kết luận

Reverse crypto không bắt đầu bằng việc thuộc lòng mọi thuật toán. Nó bắt đầu bằng khả năng đọc phép toán theo đúng độ rộng, nhận ra cấu trúc lặp, thu tham số ở runtime và chứng minh giả thuyết bằng một mô hình nhỏ có thể chạy được.

Nguồn biên soạn: “Tài liệu chuyên sâu: Mật mã học thực chiến dành cho Reverse Engineer” (Markdown do người học cung cấp). Nội dung đã được tổ chức lại và bổ sung ví dụ phục vụ học tập.